Apache Cassandra & Apache Hive - Exam Preparation Notes

PART A: APACHE CASSANDRA

1. Introduction to Cassandra

What is Apache Cassandra?

Apache Cassandra is an open-source, distributed, wide-column store NoSQL database management system designed to handle large amounts of data across many commodity servers, providing high availability with no single point of failure.

Key Features of Cassandra

| Feature | Description |

|---------|-------------|

| **Distributed Architecture** | Data is distributed across multiple nodes; no master-slave architecture |

| **High Availability** | No single point of failure; continuous availability for critical applications |

| **Scalability** | Linear scalability; add nodes without downtime |

| **Fault Tolerance** | Automatic replication; data replicated to multiple nodes |

| **Tunable Consistency** | Choose between consistency and availability per operation |

| **Fast Writes** | Excellent write performance; optimized for write-heavy workloads |

| **Flexible Schema** | Dynamic schema changes without downtime |

| **CQL (Cassandra Query Language)** | SQL-like syntax for easier adoption |

| **Peer-to-Peer Architecture** | All nodes are equal; any node can accept read/write requests |

| **Decentralized** | No central coordinator; all nodes can act as coordinators |

Cassandra Use Cases

- Time-series data (IoT, metrics, logs)
- Messaging systems (message persistence)
- E-commerce (shopping carts, user sessions)
- Social media (feeds, likes, comments)
- Real-time analytics
- Content management systems

2. CQL Data Types

Native Data Types

| Category | Data Types | Description |

|----------|------------|-------------|

| **Text/Character** | `ascii`, `text`, `varchar` | String data (UTF-8 encoded) |

| **Numeric** | `bigint`, `int`, `smallint`, `tinyint`, `varint` | Integer types |

| **Floating** | `float`, `double`, `decimal` | Floating-point numbers |

| **Boolean** | `boolean` | true/false values |

| **Binary** | `blob` | Binary large objects |

| **UUID** | `uuid`, `timeuuid` | Unique identifiers |

| **Time** | `timestamp`, `date`, `time`, `duration` | Date and time values |

| **Network** | `inet` | IP addresses (IPv4/IPv6) |

| **Counter** | `counter` | Increment/decrement values |

Complex Data Types (Collections)

| Type | Syntax | Description |

|------|--------|-------------|

| **List** | `list<type>` | Ordered collection of elements |

| **Set** | `set<type>` | Unordered collection of unique elements |

| **Map** | `map<key_type, value_type>` | Key-value pairs |

| **Tuple** | `tuple<type1, type2, ...>` | Fixed-size ordered set of elements |

Examples:

```
-- Native types
CREATE TABLE users (
user_id uuid PRIMARY KEY,
name text,
age int,
email text,
is_active boolean,
profile_image blob,
last_login timestamp,
ip_address inet
);

-- Collection types
CREATE TABLE products (
product_id uuid PRIMARY KEY,
name text,
tags set<text>, -- Unique tags
```

```
reviews list<text>, -- Ordered reviews
attributes map<text,text> -- Key-value attributes
);
```

3. CQLSH (Cassandra Query Language Shell)

What is CQLSH?

CQLSH is the command-line interface for interacting with Cassandra using CQL. It's a Python-based shell that comes bundled with Cassandra.

Starting CQLSH

```
# Default connection (localhost:9042)
cqlsh

# Connect to specific host and port
cqlsh 192.168.1.100 9042

# With username and password
cqlsh -u username -p password

# With keyspace
cqlsh -k mykeyspace
```

Important CQLSH Commands

| Command | Description |

|---------|-------------|

| `DESCRIBE keyspaces;` | List all keyspaces |

| `DESCRIBE tables;` | List all tables in current keyspace |

| `DESCRIBE table_name;` | Show table schema |

| `USE keyspace_name;` | Switch to a keyspace |

| `CONSISTENCY;` | Show current consistency level |

| `CONSISTENCY ONE;` | Set consistency level |

| `HELP;` | Show available commands |

| `EXIT;` or `QUIT;` | Exit CQLSH |

| `COPY table TO 'file.csv';` | Export data to CSV |

| `COPY table FROM 'file.csv';` | Import data from CSV |

| `TRACING ON;` | Enable query tracing |

| `PAGING OFF;` | Disable paging for large results |

4. Keyspaces

What is a Keyspace?

A keyspace is the top-level namespace in Cassandra that defines how data is replicated across nodes. It's similar to a database in relational databases.

Keyspace Components

| Component | Purpose |

|-----------|---------|

| **Replication Factor** | Number of copies of data |

| **Replication Strategy** | How replicas are placed |

| **Durable Writes** | Whether writes are written to commit log |

Creating Keyspaces

```
-- SimpleStrategy (for single data center)
CREATE KEYSPACE mykeyspace
WITH REPLICATION = {
'class': 'SimpleStrategy',
'replication_factor': 3
};

-- NetworkTopologyStrategy (for multiple data centers)
CREATE KEYSPACE mykeyspace
WITH REPLICATION = {
'class': 'NetworkTopologyStrategy',
'dc1': 3,
'dc2': 2
};

-- With durable writes disabled (faster but less safe)
CREATE KEYSPACE mykeyspace
WITH REPLICATION = {
'class': 'SimpleStrategy',
'replication_factor': 3
}
AND DURABLE_WRITES = false;
```

Keyspace Operations

```
-- Switch to keyspace
USE mykeyspace;

-- Alter keyspace
ALTER KEYSPACE mykeyspace
WITH REPLICATION = {
'class': 'SimpleStrategy',
'replication_factor': 2
};

-- Drop keyspace
DROP KEYSPACE IF EXISTS mykeyspace;

-- Describe keyspace
```

```
DESCRIBE KEYSPACE mykeyspace;
```

5. CRUD Operations

CREATE (Insert Data)

```
-- Basic INSERT
INSERT INTO users (user_id, name, email, age)
VALUES (uuid(), 'John Doe', 'john@example.com', 25);

-- Insert with TTL (Time To Live)
INSERT INTO users (user_id, name, email)
VALUES (uuid(), 'Temp User', 'temp@example.com')
USING TTL 3600; -- Expires in 1 hour

-- Insert with timestamp
INSERT INTO users (user_id, name)
VALUES (uuid(), 'User')
USING TIMESTAMP 1234567890;

-- Insert JSON
INSERT INTO users JSON '{"user_id": "uuid", "name": "John"}';
```

READ (Select Data)

```
-- Select all columns
SELECT * FROM users;

-- Select specific columns
SELECT name, email FROM users;

-- Select with WHERE clause (partition key required)
SELECT * FROM users WHERE user_id = uuid();

-- Select with multiple conditions
SELECT * FROM products WHERE category = 'Electronics' AND price > 100;

-- Select with LIMIT
SELECT * FROM users LIMIT 10;

-- Select with ALLOW FILTERING (not recommended for production)
SELECT * FROM users WHERE age > 25 ALLOW FILTERING;

-- Select with ORDER BY
SELECT * FROM events WHERE user_id = uuid() ORDER BY event_time DESC;
```

UPDATE (Modify Data)

```
-- Basic UPDATE
UPDATE users SET name = 'Jane Doe' WHERE user_id = uuid();

-- Update multiple columns
UPDATE users
SET name = 'John Smith', email = 'john.smith@example.com'
WHERE user_id = uuid();

-- Update with TTL
UPDATE users USING TTL 86400
SET status = 'temporary'
```

```
WHERE user_id = uuid();

-- Update collection (add element)
UPDATE products
SET tags = tags + {'new-tag'}
WHERE product_id = uuid();

-- Update map
UPDATE users
SET settings = settings + {'theme': 'dark'}
WHERE user_id = uuid();
```

DELETE (Remove Data)

```
-- Delete entire row
DELETE FROM users WHERE user_id = uuid();

-- Delete specific columns
DELETE email, phone FROM users WHERE user_id = uuid();

-- Delete with timestamp condition
DELETE FROM users WHERE user_id = uuid() IF EXISTS;

-- Delete from collection (remove element)
UPDATE products
SET tags = tags - {'old-tag'}
WHERE product_id = uuid();

-- Delete with TTL already set
-- (Data automatically expires after TTL)

-- Truncate table (delete all rows)
TRUNCATE users;
```

6. Collections

List Operations

```
-- Create table with list
CREATE TABLE songs (
song_id uuid PRIMARY KEY,
title text,
artists list<text>
);

-- Append element (adds to end)
UPDATE songs SET artists = artists + ['New Artist'] WHERE song_id = uuid();

-- Prepend element (adds to beginning)
UPDATE songs SET artists = ['First Artist'] + artists WHERE song_id = uuid();

-- Remove element
UPDATE songs SET artists = artists - ['Artist To Remove'] WHERE song_id = uuid();

-- Set entire list
UPDATE songs SET artists = ['Artist1', 'Artist2'] WHERE song_id = uuid();

-- Query list element
SELECT artists[0] FROM songs WHERE song_id = uuid();
```

Set Operations

```
-- Create table with set
CREATE TABLE products (
product_id uuid PRIMARY KEY,
name text,
tags set<text>
);

-- Add element (unordered, unique)
UPDATE products SET tags = tags + {'electronics'} WHERE product_id = uuid();

-- Add multiple elements
UPDATE products SET tags = tags + {'electronics', 'gadgets'} WHERE product_id = uuid();

-- Remove element
UPDATE products SET tags = tags - {'old-tag'} WHERE product_id = uuid();

-- Query contains
SELECT * FROM products WHERE tags CONTAINS 'electronics';
```

Map Operations

```
-- Create table with map
CREATE TABLE users (
user_id uuid PRIMARY KEY,
name text,
preferences map<text, text>
);

-- Add/Update key-value pair
UPDATE users
SET preferences = preferences + {'theme': 'dark'}
WHERE user_id = uuid();

-- Remove key-value pair
UPDATE users
SET preferences = preferences - {'theme'}
WHERE user_id = uuid();

-- Update single entry
UPDATE users
SET preferences['language'] = 'en'
WHERE user_id = uuid();

-- Query by map value
SELECT * FROM users WHERE preferences CONTAINS KEY 'theme';
```

7. Counter

What are Counters?

Counters are special columns used to store 64-bit signed integers that support increment and decrement operations. They're useful for tracking counts (likes, views, votes).

Counter Characteristics

- Can only be incremented or decremented

- Cannot be mixed with regular columns in the same table (except primary key)
- Initially NULL, treated as 0
- Not idempotent by default (use carefully)

Creating Counter Table

```
CREATE TABLE page_views (
page_id uuid PRIMARY KEY,
views counter,
unique_visitors counter
);
```

Counter Operations

```
-- Increment counter
UPDATE page_views
SET views = views + 1
WHERE page_id = uuid();

-- Decrement counter
UPDATE page_views
SET views = views - 1
WHERE page_id = uuid();

-- Increment by specific value
UPDATE page_views
SET views = views + 5
WHERE page_id = uuid();

-- Select counter value
SELECT views FROM page_views WHERE page_id = uuid();
```

Important Notes

```
-- WRONG: Cannot use regular columns with counters
CREATE TABLE wrong (
id uuid PRIMARY KEY,
name text, -- NOT ALLOWED
count counter
);

-- CORRECT: Only primary key and counters
CREATE TABLE correct (
id uuid PRIMARY KEY,
count1 counter,
count2 counter
);
```

8. Time To Live (TTL)

What is TTL?

TTL (Time To Live) specifies how long data should exist before being automatically deleted. Cassandra automatically removes expired data during compaction.

TTL Characteristics

- Specified in seconds
- NULL means no expiration
- Works at row or column level
- Can be set on INSERT or UPDATE
- Background deletion during compaction

Setting TTL

```
-- Insert with TTL (1 hour = 3600 seconds)
INSERT INTO users (user_id, name, session_token)
VALUES (uuid(), 'Temp User', 'abc123')
USING TTL 3600;

-- Update with TTL (24 hours)
UPDATE users USING TTL 86400
SET status = 'temporary'
WHERE user_id = uuid();

-- Set TTL on existing row
UPDATE users USING TTL 604800 -- 7 days
SET cache_data = 'data'
WHERE user_id = uuid();
```

Checking TTL

```
-- Get remaining TTL (in seconds)
SELECT TTL(session_token) FROM users WHERE user_id = uuid();

-- Get TTL for specific column
SELECT name, email, TTL(email) as email_ttl FROM users WHERE user_id = uuid();
```

Table-Level Default TTL

```
-- Set default TTL for entire table
ALTER TABLE users WITH default_time_to_live = 86400;

-- Now all inserts automatically have 24-hour TTL
INSERT INTO users (user_id, name) VALUES (uuid(), 'User');
```

Common TTL Use Cases

- Session data (expire after inactivity)
- Temporary codes (OTP, verification)
- Cache data (auto-expire old entries)
- Time-series data (retention policy)
- Temporary locks

9. Alter Commands

Alter Table Operations

```
-- Add new column
ALTER TABLE users ADD phone text;

-- Add multiple columns
ALTER TABLE users ADD (phone text, address text, age int);

-- Drop column
ALTER TABLE users DROP phone;

-- Rename column (same type only)
ALTER TABLE users RENAME email TO email_address;

-- Change column type (with restrictions)
ALTER TABLE users ALTER age TYPE bigint;

-- Add column with collection type
ALTER TABLE users ADD interests set<text>;

-- Add static column (shared across partition)
ALTER TABLE orders ADD customer_name text STATIC;
```

Alter Keyspace Operations

```
-- Change replication factor
ALTER KEYSPACE mykeyspace
WITH REPLICATION = {
'class': 'SimpleStrategy',
'replication_factor': 3
};

-- Change durable writes
ALTER KEYSPACE mykeyspace WITH DURABLE_WRITES = true;
```

Alter Table Properties

```
-- Set compaction strategy
ALTER TABLE users
WITH compaction = {
'class': 'LeveledCompactionStrategy',
'sstable_size_in_mb': 160
};

-- Set compression
ALTER TABLE users
WITH compression = {
'class': 'LZ4Compressor',
'chunk_length_in_kb': 64
};

-- Set read repair chance
ALTER TABLE users WITH read_repair_chance = 0.1;

-- Enable/disable compaction
ALTER TABLE users WITH compaction_enabled = false;
```

10. Import and Export

COPY Command (CQLSH)

Export Data

```
-- Export entire table to CSV
COPY users TO '/path/to/users.csv';

-- Export with specific columns
COPY users (user_id, name, email) TO '/path/to/users.csv';

-- Export with header row
COPY users TO '/path/to/users.csv' WITH HEADER = true;

-- Export with delimiter
COPY users TO '/path/to/users.csv' WITH DELIMITER = '|';

-- Export specific rows
COPY users TO '/path/to/users.csv' WHERE age > 25;

-- Export with NULL representation
COPY users TO '/path/to/users.csv' WITH NULL = 'NULL';
```

Import Data

```
-- Import from CSV
COPY users FROM '/path/to/users.csv';

-- Import with header
COPY users FROM '/path/to/users.csv' WITH HEADER = true;

-- Import with custom delimiter
COPY users FROM '/path/to/users.csv' WITH DELIMITER = ';';

-- Skip bad rows during import
COPY users FROM '/path/to/users.csv' WITH SKIPROWS = 1;

-- Batch size for large imports
COPY users FROM '/path/to/users.csv' WITH BATCHSIZE = 1000;

-- Import with rate limiting
COPY users FROM '/path/to/users.csv' WITH RATEFILE = '/path/to/rate';
```

SSTable Loader (Bulk Loading)

```
# For large datasets, use sstableloader
sstableloader -d 192.168.1.100 /path/to/sstables/

# With specific keyspace
sstableloader -d 192.168.1.100 keyspace/table/
```

cqlsh Commands

```
# Execute CQL file
cqlsh -e "SOURCE '/path/to/script.cql'"

# Execute and capture output
cqlsh -e "SELECT * FROM mykeyspace.users" > output.txt
```

```
# Import from file
cqlsh -f script.cql
```

11. Querying System Tables

What are System Tables?

System tables store metadata about the cluster, keyspaces, tables, and runtime statistics. They're located in system keyspaces.

System Keyspaces

| Keyspace | Purpose |

|----------|---------|

| `system` | Local node information, cluster topology |

| `system_schema` | Schema definitions (tables, columns, types) |

| `system_auth` | Authentication and authorization data |

| `system_distributed` | Repair history, view build status |

| `system_traces` | Query tracing data |

Common System Tables

```
-- List all system keyspaces
SELECT keyspace_name FROM system_schema.keyspaces;

-- List all tables in current keyspace
SELECT table_name FROM system_schema.tables WHERE keyspace_name = 'mykeyspace';

-- Get table schema
SELECT column_name, type, kind
FROM system_schema.columns
WHERE keyspace_name = 'mykeyspace' AND table_name = 'users';

-- List all columns
SELECT * FROM system_schema.columns WHERE table_name = 'users';

-- Get table properties
SELECT * FROM system_schema.tables
WHERE keyspace_name = 'mykeyspace' AND table_name = 'users';

-- View cluster information
SELECT * FROM system.local; -- Current node
SELECT * FROM system.peers; -- Other nodes

-- Get token ring (partition distribution)
SELECT * FROM system.token_ring;

-- View running compactions
SELECT * FROM system.compactions_in_progress;

-- View compaction history
SELECT * FROM system.compaction_history;

-- View available compaction strategies
```

```
SELECT * FROM system_schema.types WHERE keyspace_name = 'system';
```

Useful System Queries

```
-- Check schema version
SELECT schema_version FROM system.local;

-- Get cluster name
SELECT cluster_name FROM system.local;

-- Get release version
SELECT release_version FROM system.local;

-- Get native protocol version
SELECT native_protocol_version FROM system.local;

-- Get data center and rack
SELECT data_center, rack FROM system.local;

-- List all peer nodes
SELECT peer, data_center, rack, tokens FROM system.peers;

-- Check recent queries (if tracing enabled)
SELECT * FROM system_traces.sessions;
SELECT * FROM system_traces.events;

-- Get partition information
SELECT keyspace_name, table_name, partition_key
FROM system_schema.columns
WHERE kind = 'partition_key';

-- Get clustering columns
SELECT keyspace_name, table_name, column_name
FROM system_schema.columns
WHERE kind = 'clustering';
```

PART B: APACHE HIVE

1. Introduction to Hive

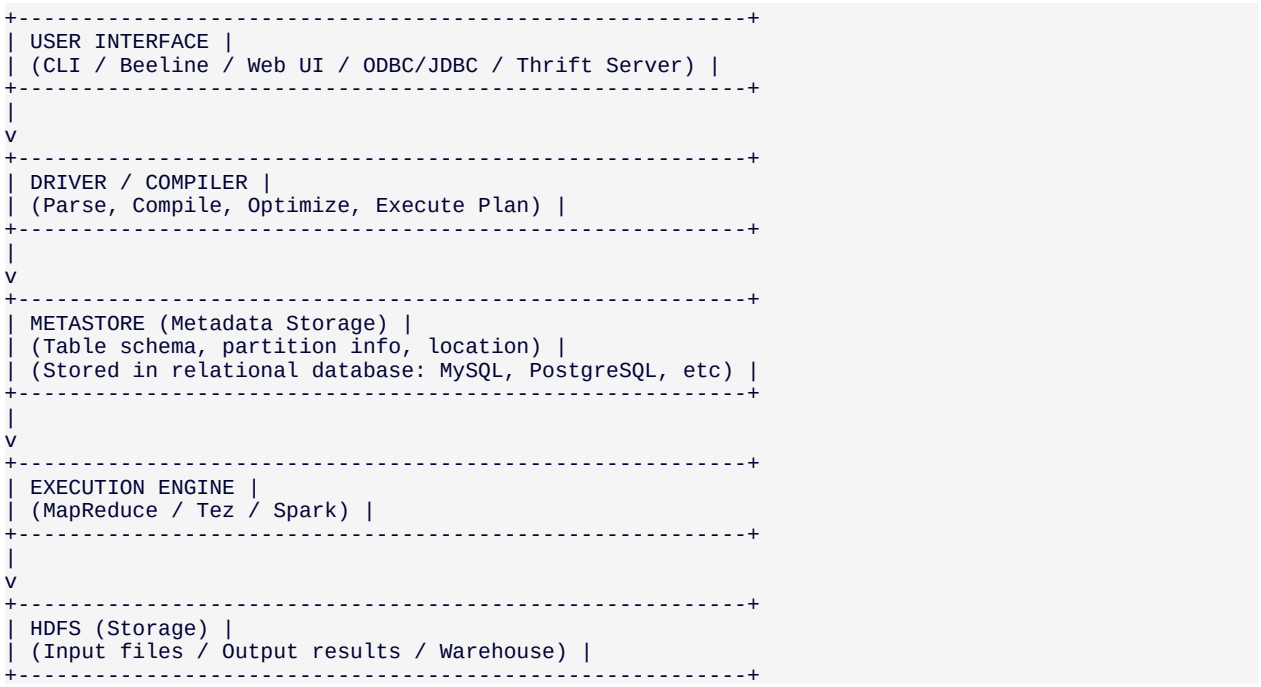
What is Apache Hive?

Apache Hive is a data warehouse software built on top of Hadoop that facilitates reading, writing, and managing large datasets residing in distributed storage using SQL-like queries (HiveQL).

Key Characteristics

- Built on Hadoop ecosystem (HDFS + MapReduce/Tez/Spark)
- Translates SQL-like queries to MapReduce jobs
- Schema-on-read (schema applied when data is read, not written)
- Optimized for batch processing, not OLTP
- Handles petabyte-scale data

Hive Architecture Components



Hive Architecture Layers

| Component | Function |

|---|---|

| **User Interface** | Submit queries (CLI, Web UI, JDBC/ODBC) |

| **Driver** | Receives queries, manages lifecycle |

| **Compiler** | Parses query, generates execution plan |

| **Optimizer** | Optimizes query plan (predicate pushdown, etc.) |

| **Executor** | Executes tasks on Hadoop |

| **Metastore** | Stores metadata about tables, columns, partitions |

| **Storage** | HDFS for actual data storage |

Hive vs Traditional RDBMS

| Feature | Hive | RDBMS |

|---|---|---|

| **Data Size** | Petabytes | Terabytes |

| **Latency** | High (minutes to hours) | Low (milliseconds) |

| **Transactions** | Limited ACID support | Full ACID |

| **Schema** | Schema-on-read | Schema-on-write |

| **Indexing** | Limited | Extensive |

| **Use Case** | Batch analytics | OLTP/OLAP |

| **Scale** | Horizontal (add nodes) | Vertical (bigger machines) |

2. Hive Data Types

Primitive Data Types

| Type | Description | Example |

|------|-------------|---------|

| `TINYINT` | 1-byte signed integer | 100 |

| `SMALLINT` | 2-byte signed integer | 1000 |

| `INT` | 4-byte signed integer | 100000 |

| `BIGINT` | 8-byte signed integer | 10000000000 |

| `FLOAT` | Single precision | 3.14 |

| `DOUBLE` | Double precision | 3.14159265359 |

| `DECIMAL(p,s)` | Arbitrary precision | DECIMAL(10,2) |

| `STRING` | Variable-length character | 'Hello World' |

| `VARCHAR(n)` | Variable-length with limit | VARCHAR(100) |

| `CHAR(n)` | Fixed-length character | CHAR(10) |

| `BOOLEAN` | true/false | true |

| `TIMESTAMP` | Date and time | '2024-01-15 10:30:00' |

| `DATE` | Date only | '2024-01-15' |

| `BINARY` | Byte array | BINARY '...' |

Complex Data Types

| Type | Description | Example |

|------|-------------|---------|

| `ARRAY<type>` | Ordered collection | ARRAY<INT> [1, 2, 3] |

| `MAP<K,V>` | Key-value pairs | MAP<STRING,INT> {'a':1, 'b':2} |

| `STRUCT<...>` | C-like structure | STRUCT<a:STRING, b:INT> |

| `UNIONTYPE<...>` | Multiple types | UNIONTYPE<INT, STRING, DOUBLE> |

Type Examples

```
-- Create table with various types
CREATE TABLE employees (
id INT,
name STRING,
salary DECIMAL(10,2),
is_active BOOLEAN,
hire_date DATE,
last_login TIMESTAMP,
skills ARRAY<STRING>,
attributes MAP<STRING, STRING>,
address STRUCT<street:STRING, city:STRING, zip:INT>
);

-- Insert data with complex types
INSERT INTO employees VALUES (
1,
'John Doe',
75000.00,
true,
'2020-01-15',
'2024-01-15 10:30:00',
ARRAY['Java', 'Python', 'SQL'],
MAP('dept', 'Engineering', 'level', 'Senior'),
NAMED_STRUCT('street', '123 Main St', 'city', 'NYC', 'zip', 10001)
);
```

Type Conversion

```
-- Implicit conversion (CAST)
SELECT CAST(salary AS STRING) FROM employees;
SELECT CAST('123' AS INT) + 100;

-- Type casting in expressions
SELECT name, CAST(hire_date AS STRING) AS hire_date_str FROM employees;

-- Handle NULLs
SELECT COALESCE(CAST(phone AS STRING), 'N/A') FROM employees;
```

3. Hive File Formats

Supported File Formats

| Format | Description | Use Case |

|--------|-------------|----------|

| **TEXTFILE** | Plain text, delimited | Default, human-readable |

| **SEQUENCEFILE** | Binary key/value format | Hadoop native, splittable |

| **RCFILE** | Record Columnar File | Column-oriented storage |

| **ORC** | Optimized Row Columnar | High compression, performance |

| **PARQUET** | Columnar, binary | Cross-platform, efficient |

| **AVRO** | Row-based, schema evolution | Schema evolution required |

TEXTFILE (Default)

```
-- Create table with TEXTFILE
CREATE TABLE users_text (
id INT,
name STRING,
email STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

-- Load data
LOAD DATA INPATH '/path/to/users.csv' INTO TABLE users_text;
```

RCFILE (Record Columnar File)

```
-- Create table with RCFILE
CREATE TABLE users_rc (
id INT,
name STRING,
email STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS RCFILE;

-- RCFILE properties
ALTER TABLE users_rc SET FILEFORMAT RCFILE;
```

ORC (Optimized Row Columnar)

```
-- Create table with ORC (recommended)
CREATE TABLE users_orc (
id INT,
name STRING,
email STRING
)
STORED AS ORC
TBLPROPERTIES (
'orc.compress'='ZLIB', -- Compression codec
'orc.stripe.size'='268435456', -- 256MB stripe size
'orc.row.index.stride'='10000' -- Index every 10000 rows
);

-- ORC with Snappy compression
CREATE TABLE users_orc_snappy (
id INT,
name STRING
)
STORED AS ORC
TBLPROPERTIES ('orc.compress'='SNAPPY');
```

PARQUET

```
-- Create table with PARQUET
CREATE TABLE users_parquet (
id INT,
name STRING,
email STRING
)
STORED AS PARQUET;

-- Parquet with compression
```

```
CREATE TABLE users_parquet_gzip (
id INT,
name STRING
)
STORED AS PARQUET
TBLPROPERTIES ('parquet.compression'='GZIP');
```

AVRO

```
-- Create table with AVRO (requires schema)
CREATE TABLE users_avro
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.avro.AvroSerDe'
STORED AS INPUTFORMAT 'org.apache.hadoop.hive.ql.io.avro.AvroContainerInputFormat'
OUTPUTFORMAT 'org.apache.hadoop.hive.ql.io.avro.AvroContainerOutputFormat';
```

Format Comparison

| Format | Compression | Query Performance | Write Speed | Splittable |

|--------|-------------|-------------------|-------------|------------|

| TEXTFILE | Low | Slow | Fast | Yes |

| SEQUENCEFILE | Medium | Medium | Medium | Yes |

| RCFILE | Good | Good | Medium | Yes |

| ORC | Excellent | Excellent | Medium | Yes |

| PARQUET | Excellent | Excellent | Medium | Yes |

| AVRO | Good | Good | Fast | Yes |

4. Hive Query Language (HQL)

DDL - Data Definition Language

Create Database

```
-- Create database
CREATE DATABASE IF NOT EXISTS sales_db
COMMENT 'Database for sales data'
LOCATION '/user/hive/warehouse/sales.db'
WITH DBPROPERTIES ('creator'='admin', 'created_date'='2024-01-15');

-- Use database
USE sales_db;

-- Show databases
SHOW DATABASES;
SHOW DATABASES LIKE 'sales*';

-- Describe database
DESCRIBE DATABASE sales_db;
DESCRIBE DATABASE EXTENDED sales_db; -- More details
```

```
-- Drop database
DROP DATABASE IF EXISTS sales_db;
DROP DATABASE IF EXISTS sales_db CASCADE; -- Drop tables too
```

Create Table

```
-- Basic table creation
CREATE TABLE IF NOT EXISTS employees (
emp_id INT COMMENT 'Employee ID',
first_name STRING COMMENT 'First name',
last_name STRING,
salary DECIMAL(10,2),
hire_date DATE,
dept_id INT
)
COMMENT 'Employee information table'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

-- Table with partition
CREATE TABLE sales (
order_id INT,
amount DECIMAL(10,2),
product STRING
)
PARTITIONED BY (year INT, month INT)
STORED AS ORC;

-- Table with bucketing
CREATE TABLE bucketed_users (
id INT,
name STRING,
email STRING
)
CLUSTERED BY (id) INTO 4 BUCKETS
STORED AS ORC;

-- External table (data not managed by Hive)
CREATE EXTERNAL TABLE external_logs (
log_time TIMESTAMP,
level STRING,
message STRING
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t'
LOCATION '/user/logs/';

-- Table with complex types
CREATE TABLE complex_data (
id INT,
tags ARRAY<STRING>,
metadata MAP<STRING, STRING>,
coordinates STRUCT<x:DOUBLE, y:DOUBLE>
);
```

Alter Table

```
-- Rename table
ALTER TABLE employees RENAME TO staff;

-- Add column
ALTER TABLE employees ADD COLUMNS (phone STRING);

-- Add multiple columns
ALTER TABLE employees ADD COLUMNS (
address STRING,
city STRING,
country STRING
);
```

```
-- Change column type
ALTER TABLE employees CHANGE COLUMN salary salary DECIMAL(12,2);

-- Rename column
ALTER TABLE employees CHANGE COLUMN first_name fname STRING;

-- Replace all columns (be careful!)
ALTER TABLE employees REPLACE COLUMNS (
emp_id INT,
full_name STRING,
annual_salary DECIMAL(12,2)
);

-- Set table properties
ALTER TABLE employees SET TBLPROPERTIES ('comment'='Updated employee table');
ALTER TABLE employees SET TBLPROPERTIES ('orc.compress'='SNAPPY');

-- Enable/disable protection
ALTER TABLE employees ENABLE NO_DROP; -- Prevent dropping
ALTER TABLE employees DISABLE NO_DROP; -- Allow dropping
```

Drop/Truncate Table

```
-- Drop table (removes metadata and data for managed tables)
DROP TABLE IF EXISTS employees;
DROP TABLE IF EXISTS employees PURGE; -- Skip trash

-- Truncate table (removes data only)
TRUNCATE TABLE employees;

-- Truncate specific partition
TRUNCATE TABLE sales PARTITION (year=2023, month=12);
```

DML - Data Manipulation Language

Load Data

```
-- Load from HDFS file
LOAD DATA INPATH '/user/data/employees.csv' INTO TABLE employees;

-- Load from local file
LOAD DATA LOCAL INPATH '/home/user/data/employees.csv' INTO TABLE employees;

-- Load and overwrite existing data
LOAD DATA INPATH '/user/data/new_employees.csv' OVERWRITE INTO TABLE employees;

-- Load into specific partition
LOAD DATA INPATH '/user/data/sales_2023.csv' INTO TABLE sales PARTITION (year=2023);
```

Insert Data

```
-- Insert single row
INSERT INTO employees VALUES (1, 'John', 'Doe', 75000.00, '2020-01-15', 10);

-- Insert with column list
INSERT INTO employees (emp_id, first_name, last_name) VALUES (2, 'Jane', 'Smith');

-- Insert from SELECT
INSERT INTO employees
SELECT * FROM staging_employees WHERE status = 'active';

-- Insert overwrite (replaces all data)
INSERT OVERWRITE TABLE employees
SELECT * FROM staging_employees;
```

```
-- Insert into specific partition
INSERT INTO sales PARTITION (year=2024, month=1)
SELECT order_id, amount, product FROM staging_sales WHERE year=2024 AND month=1;

-- Insert multiple tables (multi-insert)
FROM staging_data
INSERT INTO table1 SELECT col1, col2 WHERE type='A'
INSERT INTO table2 SELECT col1, col3 WHERE type='B';

-- Dynamic partition insert
INSERT INTO sales PARTITION (year, month)
SELECT order_id, amount, product, year, month FROM staging_sales;
```

SELECT Queries

```
-- Basic SELECT
SELECT * FROM employees;
SELECT emp_id, first_name, last_name FROM employees;

-- SELECT with WHERE
SELECT * FROM employees WHERE salary > 50000;
SELECT * FROM employees WHERE dept_id = 10 AND hire_date > '2020-01-01';

-- SELECT with ORDER BY
SELECT * FROM employees ORDER BY salary DESC;
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
ORDER BY avg_salary DESC;

-- SELECT with LIMIT
SELECT * FROM employees LIMIT 10;
SELECT * FROM employees WHERE salary > 100000 LIMIT 5;

-- SELECT with aggregation
SELECT
dept_id,
COUNT(*) AS emp_count,
AVG(salary) AS avg_salary,
MAX(salary) AS max_salary,
MIN(salary) AS min_salary,
SUM(salary) AS total_salary
FROM employees
GROUP BY dept_id;

-- SELECT with HAVING
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 60000;

-- SELECT with JOIN
SELECT e.emp_id, e.first_name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;

-- LEFT JOIN
SELECT e.emp_id, e.first_name, COALESCE(d.dept_name, 'Unknown') AS dept
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;

-- Subqueries
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Correlated subquery
SELECT e.emp_id, e.first_name
FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id);

-- SELECT with CASE
```

```
SELECT
first_name,
salary,
CASE
WHEN salary < 50000 THEN 'Low'
WHEN salary BETWEEN 50000 AND 100000 THEN 'Medium'
ELSE 'High'
END AS salary_level
FROM employees;

-- SELECT with complex types
SELECT name, skills[0] AS primary_skill FROM employees;
SELECT name, metadata['department'] AS dept FROM employees;
SELECT name, coordinates.x, coordinates.y FROM locations;

-- SELECT with LATERAL VIEW (explode arrays)
SELECT name, skill
FROM employees
LATERAL VIEW EXPLODE(skills) t AS skill;

-- SELECT with LATERAL VIEW for maps
SELECT name, attr_key, attr_value
FROM employees
LATERAL VIEW EXPLODE(metadata) t AS attr_key, attr_value;
```

Advanced HQL

Window Functions

```
-- ROW_NUMBER
SELECT
emp_id,
first_name,
salary,
ROW_NUMBER() OVER (ORDER BY salary DESC) AS rank
FROM employees;

-- RANK (handles ties)
SELECT
emp_id,
first_name,
salary,
RANK() OVER (ORDER BY salary DESC) AS rank
FROM employees;

-- DENSE_RANK
SELECT
emp_id,
first_name,
salary,
DENSE_RANK() OVER (ORDER BY salary DESC) AS rank
FROM employees;

-- PARTITION BY
SELECT
emp_id,
first_name,
dept_id,
salary,
ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank
FROM employees;

-- LEAD/LAG
SELECT
emp_id,
first_name,
salary,
LAG(salary, 1) OVER (ORDER BY emp_id) AS prev_salary,
LEAD(salary, 1) OVER (ORDER BY emp_id) AS next_salary
```

```
FROM employees;

-- SUM/AVG as window functions
SELECT
emp_id,
first_name,
salary,
SUM(salary) OVER (ORDER BY emp_id) AS running_total,
AVG(salary) OVER () AS overall_avg
FROM employees;
```

Common Table Expressions (CTE)

```
-- Simple CTE
WITH high_earners AS (
SELECT * FROM employees WHERE salary > 100000
)
SELECT * FROM high_earners WHERE dept_id = 10;

-- Multiple CTEs
WITH
dept_stats AS (
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
),
high_depts AS (
SELECT dept_id FROM dept_stats WHERE avg_salary > 80000
)
SELECT e.* FROM employees e
JOIN high_depts h ON e.dept_id = h.dept_id;

-- Recursive CTE (for hierarchical data)
WITH RECURSIVE emp_hierarchy AS (
-- Base case: top-level employees
SELECT emp_id, name, manager_id, 0 AS level
FROM employees
WHERE manager_id IS NULL

UNION ALL

-- Recursive case
SELECT e.emp_id, e.name, e.manager_id, h.level + 1
FROM employees e
JOIN emp_hierarchy h ON e.manager_id = h.emp_id
)
SELECT * FROM emp_hierarchy;
```

5. RCFILE Implementation

What is RCFILE?

RCFile (Record Columnar File) is a data placement structure designed for MapReduce-based data warehouse systems. It combines the advantages of both row-store and column-store approaches.

RCFILE Structure

```
+-----------------------------------------------------+
| RCFILE Structure |
+-----------------------------------------------------+
```

```
| Row Group 1 |
| +-------------------------------------------------+ |
| | Sync Marker (for splitting) | |
| +-------------------------------------------------+ |
| | Row Group Header (metadata) | |
| +-------------------------------------------------+ |
| | Column 1 Data | Column 2 Data | Column 3 Data | |
| | (compressed) | (compressed) | (compressed) | |
| +-------------------------------------------------+ |
+-----------------------------------------------------+
| Row Group 2 |
| +-------------------------------------------------+ |
| | Sync Marker | |
| +-------------------------------------------------+ |
| | Row Group Header | |
| +-------------------------------------------------+ |
| | Column 1 Data | Column 2 Data | Column 3 Data | |
| +-------------------------------------------------+ |
+-----------------------------------------------------+
```

RCFILE Properties

| Property | Description | Default |

|----------|-------------|---------|

| `hive.io.rcfile.record.buffer.size` | Row group size (bytes) | 4MB |

| `hive.io.rcfile.column.number.conf` | Number of columns | - |

Creating RCFILE Tables

```
-- Create table stored as RCFILE
CREATE TABLE rc_employees (
id INT,
name STRING,
salary DECIMAL(10,2),
dept_id INT
)
STORED AS RCFILE;

-- Create RCFILE with specific properties
CREATE TABLE rc_sales (
order_id BIGINT,
customer_id STRING,
amount DECIMAL(12,2),
order_date TIMESTAMP
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t'
STORED AS RCFILE
TBLPROPERTIES (
'hive.io.rcfile.record.buffer.size'='16777216' -- 16MB
);

-- Convert existing table to RCFILE
ALTER TABLE employees SET FILEFORMAT RCFILE;

-- Insert data into RCFILE
INSERT OVERWRITE TABLE rc_employees
SELECT * FROM employees;
```

RCFILE Advantages

Column-oriented storage within row groups

• Only necessary columns are read

• Better compression for similar data types

Row group organization

• Maintains row-level relationships

• Efficient for queries needing multiple columns

Lazy decompression

• Columns decompressed only when needed

• Reduces I/O for selective queries

Block-level compression

• Each column compressed independently

• Better compression ratios

Querying RCFILE

```
-- Read specific columns (efficient)
SELECT name, salary FROM rc_employees WHERE dept_id = 10;

-- Read all columns (less efficient)
SELECT * FROM rc_employees;

-- Join with RCFILE
SELECT e.name, d.dept_name
FROM rc_employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

6. SerDe (Serializer/Deserializer)

What is SerDe?

SerDe is a combination of Serializer and Deserializer that instructs Hive on how to process a record. It handles:

- **Deserialization**: Reading data from HDFS into Hive objects
- **Serialization**: Writing Hive objects back to HDFS

Built-in SerDes

| SerDe | Purpose | Use Case |

|-------|---------|----------|

| `LazySimpleSerDe` | Default, simple delimited | CSV, TSV files |

| `OpenCSVSerde` | RFC4180 compliant CSV | Standard CSV |

| `RegexSerDe` | Regular expression parsing | Log files |

| `JsonSerDe` | JSON format | JSON data |

| `AvroSerDe` | Avro binary format | Schema evolution |

| `ParquetHiveSerDe` | Parquet files | Columnar storage |

| `OrcSerde` | ORC files | Optimized columnar |

| `HBaseSerDe` | HBase integration | Hive-HBase integration |

Using SerDe

```
-- Using OpenCSVSerde for proper CSV parsing
CREATE TABLE csv_table (
id INT,
name STRING,
email STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES (
"separatorChar" = ",",
"quoteChar" = "\"",
"escapeChar" = "\\"
)
STORED AS TEXTFILE;

-- Using RegexSerDe for log parsing
CREATE TABLE apache_logs (
host STRING,
identity STRING,
user STRING,
time STRING,
request STRING,
status STRING,
size STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.RegexSerDe'
WITH SERDEPROPERTIES (
"input.regex" = "([^ ]*) ([^ ]*) ([^ ]*) (-|\\[[^\\]]*\\]) ([^ \"]*|\"[^\"]*\") (-|[0-9]*)
(-|[0-9]*)"
)
STORED AS TEXTFILE;

-- Using JsonSerDe
CREATE TABLE json_table (
id STRING,
name STRING,
details MAP<STRING, STRING>
)
ROW FORMAT SERDE 'org.apache.hive.hcatalog.data.JsonSerDe'
STORED AS TEXTFILE;

-- Using custom delimiter
CREATE TABLE custom_delim (
col1 STRING,
col2 STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe'
WITH SERDEPROPERTIES (
'field.delim' = '\u0001', -- SOH character
'collection.delim' = ',',
'mapkey.delim' = ':'
);
```

Custom SerDe Example (Log File)

```
-- Parse Apache Combined Log Format
CREATE TABLE access_logs (
ip_address STRING,
user_identifier STRING,
user_name STRING,
timestamp STRING,
method STRING,
uri STRING,
protocol STRING,
status_code INT,
response_size BIGINT,
referer STRING,
user_agent STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.RegexSerDe'
WITH SERDEPROPERTIES (
"input.regex" = '^(\\S+) (\\S+) (\\S+) \\[([\\w\\d:/]+\\s[+\\-]\\d{4})\\] \"(\\S+) (\\S+) (\\S+)\"
(\\d{3}) (\\d+) \"(.*?)\" \"(.*?)\"'
)
STORED AS TEXTFILE;
```

7. User Defined Function (UDF)

Types of Hive Functions

| Type | Description | Example |

|------|-------------|---------|

| **UDF** | User Defined Function | One row in → one row out (scalar) |

| **UDAF** | User Defined Aggregate Function | Multiple rows → one value (aggregate) |

| **UDTF** | User Defined Table Function | One row → multiple rows (explode) |

Built-in Functions

```
-- Mathematical functions
SELECT ABS(-10); -- 10
SELECT ROUND(3.14159, 2); -- 3.14
SELECT FLOOR(3.7); -- 3
SELECT CEIL(3.2); -- 4
SELECT POWER(2, 3); -- 8
SELECT SQRT(16); -- 4
SELECT RAND(); -- Random 0-1

-- String functions
SELECT LENGTH('Hello'); -- 5
SELECT UPPER('hello'); -- HELLO
SELECT LOWER('HELLO'); -- hello
SELECT TRIM(' hello '); -- hello
SELECT SUBSTR('Hello World', 1, 5); -- Hello
SELECT CONCAT('Hello', ' ', 'World'); -- Hello World
SELECT SPLIT('a,b,c', ','); -- ["a", "b", "c"]
SELECT REGEXP_REPLACE('Hello', 'l', 'x'); -- Hexxo
SELECT REGEXP_EXTRACT('2024-01-15', '(\\d{4})', 1); -- 2024

-- Date functions
SELECT CURRENT_DATE; -- Today's date
SELECT CURRENT_TIMESTAMP; -- Current timestamp
SELECT YEAR('2024-01-15'); -- 2024
SELECT MONTH('2024-01-15'); -- 1
SELECT DAY('2024-01-15'); -- 15
```

```
SELECT DATE_ADD('2024-01-15', 7); -- 2024-01-22
SELECT DATEDIFF('2024-01-20', '2024-01-15'); -- 5
SELECT FROM_UNIXTIME(1705315200); -- Convert Unix timestamp
SELECT UNIX_TIMESTAMP('2024-01-15'); -- To Unix timestamp
SELECT DATE_FORMAT('2024-01-15', 'yyyy-MM'); -- 2024-01

-- Conditional functions
SELECT COALESCE(NULL, 'A', 'B'); -- A
SELECT IF(1=1, 'Yes', 'No'); -- Yes
SELECT CASE WHEN 1=1 THEN 'Yes' ELSE 'No' END; -- Yes
SELECT NVL(NULL, 'Default'); -- Default (similar to COALESCE)
SELECT NULLIF(1, 1); -- NULL
SELECT ISNULL(NULL); -- true

-- Collection functions
SELECT SIZE(ARRAY(1,2,3)); -- 3
SELECT ARRAY_CONTAINS(ARRAY(1,2,3), 2); -- true
SELECT SORT_ARRAY(ARRAY(3,1,2)); -- [1,2,3]
SELECT MAP_KEYS(MAP('a',1,'b',2)); -- ["a","b"]
SELECT MAP_VALUES(MAP('a',1,'b',2)); -- [1,2]
SELECT EXPLODE(ARRAY(1,2,3)); -- 3 rows: 1, 2, 3

-- Aggregate functions
SELECT COUNT(*) FROM employees;
SELECT COUNT(DISTINCT dept_id) FROM employees;
SELECT SUM(salary) FROM employees;
SELECT AVG(salary) FROM employees;
SELECT MAX(salary) FROM employees;
SELECT MIN(salary) FROM employees;
SELECT STDDEV_SAMP(salary) FROM employees; -- Sample std dev
SELECT STDDEV_POP(salary) FROM employees; -- Population std dev
SELECT VARIANCE(salary) FROM employees;
SELECT COLLECT_SET(dept_id) FROM employees; -- Array of unique values
SELECT COLLECT_LIST(dept_id) FROM employees; -- Array (allows duplicates)

-- Table generating functions
SELECT EXPLODE(ARRAY(1,2,3)) AS num;
SELECT POSEXPLODE(ARRAY('a','b')) AS (pos, val);
SELECT STACK(2, 'A', 1, 'B', 2) AS (name, value);
SELECT INLINE(ARRAY(NAMED_STRUCT('a',1,'b',2))); -- Expand structs
SELECT JSON_TUPLE('{"a":1,"b":2}', 'a', 'b'); -- Parse JSON
SELECT PARSE_URL('http://example.com:8080/path', 'HOST'); -- example.com
SELECT PARSE_URL_TUPLE('http://example.com', 'HOST', 'PATH');
```

Creating Temporary Functions

```
-- Create temporary function (session only)
ADD JAR /path/to/my-udf.jar;
CREATE TEMPORARY FUNCTION my_function AS 'com.example.hive.MyUdf';

-- Use the function
SELECT my_function(column_name) FROM table_name;

-- Drop temporary function
DROP TEMPORARY FUNCTION IF EXISTS my_function;
```

Creating Permanent Functions

```
-- Create permanent function (available to all users)
CREATE FUNCTION my_permanent_function AS 'com.example.hive.MyUdf'
USING JAR 'hdfs:///path/to/my-udf.jar';

-- Use the function
SELECT my_permanent_function(column_name) FROM table_name;

-- Show functions
SHOW FUNCTIONS LIKE 'my*';
DESCRIBE FUNCTION my_permanent_function;
```

```
DESCRIBE FUNCTION EXTENDED my_permanent_function;

-- Drop function
DROP FUNCTION IF EXISTS my_permanent_function;
```

UDF Example (Java Pseudo-code)

```
// Simple UDF that converts string to uppercase
import org.apache.hadoop.hive.ql.exec.UDF;

public class MyUpper extends UDF {
public String evaluate(String input) {
if (input == null) return null;
return input.toUpperCase();
}
}

// Usage in Hive
// CREATE TEMPORARY FUNCTION my_upper AS 'com.example.MyUpper';
// SELECT my_upper(name) FROM employees;
```

UDAF Example (Aggregate)

```
// UDAF that calculates average (simplified)
import org.apache.hadoop.hive.ql.exec.UDAF;
import org.apache.hadoop.hive.ql.exec.UDAFEvaluator;

public class MyAverage extends UDAF {
public static class AverageEvaluator implements UDAFEvaluator {
private long sum;
private long count;

public void init() {
sum = 0;
count = 0;
}

public boolean iterate(Long value) {
if (value != null) {
sum += value;
count++;
}
return true;
}

public Long terminatePartial() {
return count == 0 ? null : sum / count;
}

public boolean merge(Long other) {
// Merge partial results
return true;
}

public Long terminate() {
return count == 0 ? null : sum / count;
}
}
}
```

UDTF Example (Table Function)

```
-- UDTF that splits a delimited string into rows
-- CREATE TEMPORARY FUNCTION split_to_rows AS 'com.example.SplitToRows';
```

```
-- Usage
SELECT name, split_value
FROM data
LATERAL VIEW split_to_rows(tags, ',') t AS split_value;

-- Similar to EXPLODE but custom logic
```

QUICK REFERENCE: Important Commands

Cassandra Quick Reference

```
-- Keyspace operations
CREATE KEYSPACE ks WITH REPLICATION = {'class': 'SimpleStrategy', 'replication_factor': 3};
USE ks;
DROP KEYSPACE IF EXISTS ks;

-- Table operations
CREATE TABLE users (id UUID PRIMARY KEY, name TEXT);
ALTER TABLE users ADD email TEXT;
DROP TABLE IF EXISTS users;
TRUNCATE users;

-- CRUD
INSERT INTO users (id, name) VALUES (uuid(), 'John');
SELECT * FROM users WHERE id = ?;
UPDATE users SET name = 'Jane' WHERE id = ?;
DELETE FROM users WHERE id = ?;

-- Collections
UPDATE users SET tags = tags + {'new'} WHERE id = ?;
UPDATE users SET tags = tags - {'old'} WHERE id = ?;
UPDATE users SET attrs = attrs + {'key': 'val'} WHERE id = ?;

-- Counters
UPDATE counters SET count = count + 1 WHERE id = ?;

-- TTL
INSERT INTO users (id, name) VALUES (uuid(), 'Temp') USING TTL 3600;
SELECT TTL(name) FROM users WHERE id = ?;

-- System tables
SELECT * FROM system_schema.keyspaces;
SELECT * FROM system_schema.tables WHERE keyspace_name = 'ks';
DESCRIBE KEYSPACE ks;
DESCRIBE TABLE users;
```

Hive Quick Reference

```
-- Database
CREATE DATABASE db;
USE db;

-- Table
CREATE TABLE t (id INT, name STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS
TEXTFILE;
CREATE TABLE t_orc (id INT) STORED AS ORC;
CREATE TABLE t_part (id INT) PARTITIONED BY (year INT);

-- Data
LOAD DATA INPATH '/path' INTO TABLE t;
INSERT INTO t VALUES (1, 'A');
INSERT OVERWRITE TABLE t SELECT * FROM src;
```

```
-- Query
SELECT * FROM t WHERE id > 0;
SELECT id, COUNT(*) FROM t GROUP BY id;
SELECT a.*, b.* FROM a JOIN b ON a.id = b.id;

-- Complex types
SELECT tags[0] FROM t; -- Array access
SELECT attrs['key'] FROM t; -- Map access
SELECT coord.x FROM t; -- Struct access
LATERAL VIEW EXPLODE(tags) x AS tag; -- Explode array

-- Functions
SELECT UPPER(name), LENGTH(name) FROM t;
SELECT YEAR(date_col), MONTH(date_col) FROM t;
SELECT CASE WHEN x > 0 THEN 'Pos' ELSE 'Neg' END FROM t;

-- Window functions
SELECT name, ROW_NUMBER() OVER (ORDER BY salary DESC) FROM t;
SELECT name, SUM(salary) OVER (PARTITION BY dept) FROM t;

-- File format
ALTER TABLE t SET FILEFORMAT ORC;
ALTER TABLE t SET TBLPROPERTIES ('orc.compress'='SNAPPY');

-- SerDe
CREATE TABLE json_t (...) ROW FORMAT SERDE 'org.openx.data.jsonserde.JsonSerDe';
```

EXAM TIPS

Cassandra Exam Focus

Data Modeling: Understand partition keys, clustering columns, and compound primary keys

Consistency Levels: ONE, QUORUM, ALL, LOCAL_ONE, LOCAL_QUORUM, EACH_QUORUM

CQL Syntax: Be comfortable with all CRUD operations

Collections: Know when to use List vs Set vs Map

Counter Tables: Remember they can't mix with regular columns

TTL: Understand automatic expiration use cases

System Tables: Know how to query metadata

Hive Exam Focus

Architecture: Understand the flow from query to execution

File Formats: Know the differences between TEXTFILE, ORC, PARQUET, RCFILE

SerDe: Understand serialization/deserialization concept

Partitions and Buckets: Know when and how to use them

HQL Queries: Practice complex JOINs, subqueries, and aggregations

Window Functions: Understand ROW_NUMBER, RANK, LEAD, LAG

UDFs: Know the three types (UDF, UDAF, UDTF) and their purposes

Common Mistakes to Avoid

Cassandra: Forgetting that WHERE clause must include partition key

Cassandra: Mixing counter columns with regular columns

Hive: Using ORC/PARQUET without understanding compression benefits

Hive: Not using partitions for large time-series data

Both: Not handling NULL values properly in queries

Both: Confusing DROP TABLE (deletes data) vs TRUNCATE (keeps schema)

Good luck with your exam! ■